

构建 libterae

Makefile, 从桌面到裸板

Emilio Schininà, 自动化工程硕士
版本 0.3
2026 年 11 月 8 日

版权所有 © 2026 Emilio Schininà, 自动化工程硕士。libterae 以 GNU 通用公共许可证第 3 版或任何更高版本, 或以商业许可证发布。

目录

1 构建 **1**
1.1 五条命令 . . . 1
1.2 需要什么 . . . 1
1.3 生成什么 . . . 2
1.4 程序链接什么 . . . 2

2 变量 **3**
2.1 归档器 . . . 4
2.2 页大小 . . . 4
2.3 各张表的大小 . . . 5
2.4 构建文件夹 . . . 6

3 选择系统 **7**
3.1 从本机构建另一个系统 . . . 8

4 内存函数 **9**
4.1 各文件夹 . . . 9
4.2 `PORT=auto` 怎样决定 . . . 10
4.3 只要内存归档 . . . 11

5 带 TLS 构建 **13**
5.1 Mbed TLS . . . 13
5.2 库与程序 . . . 14
5.3 板子上的 TLS . . . 14

6 共享库 **15**
6.1 生成什么 . . . 15
6.2 程序链接什么 . . . 16

building-zh.tex Page iv November 8, 2026 13:38

6.3 版本 16
6.4 它给程序什么 17
6.5 带 TLS 17
6.6 哪里没有 17

7 嵌入式系统 19
7.1 裸机构建 19
7.2 程序提供什么 20
7.3 实时内核 20
7.4 板子提供什么 21
7.5 固件的编译选项 22

8 另一个 C 库 23
8.1 `platform/posix/` 需要什么 23
8.2 让编译器指向它 24
8.3 检查一次构建调用了什么 24

9 参考 25
9.1 目标 25
9.2 构建什么时候停下 25
9.3 在别的项目的构建里使用 27

1

构建

该库由仓库根的 `Makefile` 构建，用 GNU `make` 和一个 C99 编译器。没有配置步骤：命令行就是全部配置，写进 `build/` 下某个文件夹里的归档文件就是全部结果；要求时，还有 `lib/` 里的共享库（第 6 章）。

1.1 五条命令

```
# 本机：build/libterae.a
make
# 另一个系统：build/win32/libterae.a
make OS=win32 CC='zig cc -target x86_64-windows-gnu' AR='zig ar' PAGE_SIZE=4096
# 没有系统、没有 C 库的板子：build/none-armv7m/，两个归档
make OS=none PORT=auto CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' PAGE_SIZE=4096
# 带 TLS：build/tls/libterae.a
make WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls"
# 一次构建的全部命令，只打印不执行
make -n -B OS=posix
```

1.2 需要什么

- **一个 C99 编译器。** `cc`，除非 `CC` 指定了别的：交叉编译器，或带参数的包装命令，如 `zig cc -target x86_64-linux-musl`。
- **GNU `make`。** 3.81 版（macOS 装的就是这一版）或更高。BSD `make` 读不了这个文件。
- **面向目标的归档器。** 从 `CC` 找出来，或用 `AR` 指定，如 §2.1 所述。

别的都不要：没有 CMake，没有 pkg-config，没有脚本。只有 TLS 需要 Mbed TLS，而它自己的构建用 CMake（第 5 章）。

building-zh.tex Page 2 November 8, 2026 13:38

1.3 生成什么

- **libterae.a**。库：src/，含报文、主题、服务器和客户端，以及（除非 OS=none）platform/ 中的文件，它们把前者接到系统的套接字、时钟和 TLS 上，以及保存会话状态存储区的文件或闪存上。
- **libterae_port.a**。memcpy、memmove、memset 和 memcmp，供没有 C 库提供它们的构建使用。只有当 PORT 指定了 src/port/ 下的某个文件夹时才生成（第 4 章）。
- 共享库。用 make shared 时，该库作为程序启动时加载的共享库，放在 lib/ 里，旁边有 pkgconfig/libterae.pc（第 6 章）。

两个归档都没有 main、没有线程、也不调用分配器：src/ 用的每一个字节都由程序交给它。platform/ 有一些静态表，由第 2.3 节的那些数字决定大小，还有一个约 128 KiB 的缓冲区，供恢复和复制存储区时使用。

1.4 程序链接什么

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U main.c build/libterae.a -o program
```

程序读 include/ 的头文件，并且要按库构建时的页大小编译：MQTT_PAGE_SIZE 决定程序交给库的数组有多大，而归档并不记录它（§2.2）。构建还需要的东西都放在 libterae.a 之后：带 PORT 的构建加 libterae_port.a，带 TLS 的加 Mbed TLS 的三个归档，Windows 上加 ws2_32 和 bcrypt。

2

变量

每个选择都是命令行上的一个变量，如 `make OS=posix`。make 也从环境取它们，因此在某个 shell 里设一次就行。

表 2.1 构建的变量

变量	它选择什么
OS	platform/ 下的文件夹：posix、linux、macos、win32、freertos、ucos3，或用 none 只构建 src/。默认是本机的：Darwin 上为 macos，Linux 上为 linux，其它地方为 posix（第 3 章）。
PORT	src/port/ 下的文件夹，供没有 C 库的构建使用；或用 auto 选用与处理器相配的那一个。默认为空，把内存函数留给 C 库（第 4 章）。
CROSS_COMPILE	GNU 工具链的前缀，如 arm-none-eabi-。CC 变成该前缀加 gcc，除非也给出了 CC。
CC	编译器，连同它需要的参数。默认是 cc。
AR	归档器（§2.1）。
CFLAGS	优化、处理器和调试信息。默认是 -O2 -g。
STRICT	src/ 的语言和警告：-std=c99 -pedantic -Wall -Wextra -Werror -Wshadow -Wconversion -Wno-sign-conversion。STRICT= 把它们全部清空，供不接受其中某个选项的编译器使用。
WITH_TLS	yes 时用 Mbed TLS 构建 platform/platform_tls.c。默认的 no 也构建它，但每个函数都报告 MQTT_TLS_ERROR（第 5 章）。
MBEDTLS_PREFIX	Mbed TLS 安装所在的文件夹，供编译器自己找不到其头文件时使用。
BUILD	目标文件和归档所在的文件夹（§2.4）。

续页

building-zh.tex Page 4 November 8, 2026 13:38

表 2.1 构建的变量(续)

变量	它选择什么
PAGE_SIZE	页大小, 4096 到 65536 之间的 2 的幂(§2.2)。
RTOS_INCLUDES	实时内核、其网络栈和程序配置的 -I 选项, 供 freertos 和 ucos3 使用(§7.3)。
PLATFORM_DEFS	只给 platform/ 中的文件的定义。
WAIT_SOCKETS SOCKETS NET_EVENTS TLS_SESSIONS TLS_POOL_BYTES	platform/ 中各张表的大小(§2.3)。

2.1 归档器

AR 默认取 CC 旁边的 llvm-ar, 如果那里确实有一个; 否则取用 -print-prog-name=ar 问 CC 时它报出的那个 ar; 再否则取 ar。GNU 工具链会报出自己的归档器, 因此什么都不用给。报不出归档器的编译器, 如 zig cc, 或旁边没有 llvm-ar 的 clang, 只能落到本机的 ar, 而它写出的归档, 其索引目标的链接器可能读不了。给这样的构建一个能写目标格式的归档器:AR='zig ar', 或某个 llvm-ar。

2.2 页大小

MQTT_PAGE_SIZE 是服务器各内存区被切成的单位, 应当等于运行程序的系统的页大小。默认情况下, 构建从运行 make 的机器上取值:

- **16384**。在 arm64 上的 Darwin, 以及 loongarch64。
- **65536**。在 ppc64 和 ppc64le 上的 Linux。
- **4096**。其它一切。

交叉构建是给另一台机器的, 所以要自己给出 PAGE_SIZE: 否则在 Apple 芯片的 Mac 上, 每次构建都会得到 16384。程序要用同一个数字编译, 即以 -DMQTT_PAGE_SIZE 给出; 不带着它编译的程序得到头文件的默认值 4096。数字超出范围或不是 2 的幂时, 构建在 include/base_page.h 中停下:

building-zh.tex Page 5 November 8, 2026 13:38


```
$ make PAGE_SIZE=3000
include/base_page.h:60:2: error: "MQTT_PAGE_SIZE must be from 4096 to 65536"
```

页大小不在构建文件夹的名字里，所以用另一个页大小的构建，要给它自己的文件夹：`make PAGE_SIZE=4096 BUILD=build/page4096`。

2.3 各张表的大小

五个数字决定 `platform/` 中静态表的大小。默认值适合桌面服务器，要占几兆字节；`freertos` 和 `ucos3` 设得小一些，任何构建都可以自己设：

表 2.2 各张表的大小

变量	它决定的大小	默认值
`WAIT_SOCKETS`	`MQTT_PLATFORM_WAIT_SOCKETS`：一次等待覆盖的套接字，从 8 到 1048576。	65536；`freertos` 和 `ucos3` 上为 32
`SOCKETS`	`MQTT_PLATFORM_SOCKETS`：`win32` 和 `freertos` 的句柄表，它们的套接字不是小整数。	`WAIT_SOCKETS` 加 64；`freertos` 上为 48
`NET_EVENTS`	`MQTT_NET_EVENTS`：一次等待报告的事件数。	1024；`freertos` 和 `ucos3` 上为 32
`TLS_SESSIONS`	`MQTT_NET_TLS_SESSIONS`：同时存在的 TLS 会话数，从 1 到 65536。	1024；`freertos` 和 `ucos3` 上为 1
`TLS_POOL_BYTES`	`MQTT_TLS_POOL_BYTES`：Mbed TLS 从中取内存的静态池。	64 MiB

每一个都是给构建所编译的每个文件的定义，而它们都不改变程序看到的结构体。程序若从头文件读取这些上限之一，例如把 `max_sockets` 与 `MQTT_PLATFORM_WAIT_SOCKETS` 对照，就要用同样的定义编译。每条编译命令都带着它们，所以 `make -n` 能看出一次构建取了什么：

```
make -n -B OS=freertos RTOS_INCLUDES=-I. | grep -o -- '-DMQTT_[A-Z_]*=[0-9]*U*' \
  | sort -u
```

building-zh.tex Page 6 November 8, 2026 13:38

2.4 构建文件夹

任何方面不同的两次构建不能共用目标文件，所以 BUILD 默认是 build/ 下的一个文件夹，名字由使这次构建与众不同的东西组成：WITH_TLS=yes 时是 tls，然后（当 OS 不是本机的时候）是 OS，然后 PORT 所取的文件夹，用连字符连起来。

```
make                                          # build/
make WITH_TLS=yes                             # build/tls/
make OS=win32 CC=...                          # build/win32/
make OS=none PORT=auto CROSS_COMPILE=arm-none-eabi- CFLAGS='-mcpu=cortex-m4 ...'
                                              # build/none-armv7m/
make WITH_TLS=yes OS=freertos PORT=armv7m ... # build/tls-freertos-armv7m/
```

CC、CFLAGS、PAGE_SIZE 和各表大小不在名字里，所以两次 Linux 构建，一次用 glibc、一次用 musl，都落到 build/linux/。这没问题，因为第二次会把所有东西重建一遍；但要两者都留着，就各给一个 BUILD。

构建在文件夹里保留一个文件，名字是 flags- 后接编译器与每个选项的校验和。其中任何一个变了，名字就变，文件夹里的一切都重新构建，因此换了 CFLAGS 不需要 make clean。就地升级、名字不变的工具链不会被察觉：用 make -B 构建一次即可。

make clean 删掉 BUILD，以及 make shared 会用的 lib/ 下的文件夹；给了 BUILD 时，后者就在 BUILD 里面。不带变量时那就是整个 build/ 和 lib/；make clean OS=win32 只删 build/win32/ 和 lib/win32/，而树外的 BUILD 在它所在处删除。传给 make 的 LIBDIR 从不删除。

3

选择系统

与系统打交道的 platform/ 文件夹由 OS 选择。每个系统都构建 platform/ 顶层所有系统共用的六个文件（platform_net_server.c、platform_net_store.c、platform_net_client.c、platform_net_runs.c、platform_tls.c 和 platform_parse.c）、它所用的等待、句柄与存储区的共用文件，以及它自己的文件夹。除两个实时系统外，每个系统都通过 platform_store_file.c 把存储区保存在文件中；两个实时系统则通过 platform_store_flash.c 保存在闪存中。每个文件夹用自己的定义编译，所以 posix/ 只看到 POSIX，即使在 macOS 构建里也是如此；platform/ 顶层则完全不包含系统头文件。

表 3.1 各系统

OS	它构建什么
macos	套接字、时钟和文件用 posix/，等待用 macos/ 中的 kqueue，同步用能到达磁盘自己缓存的 F_FULLFSYNC；macos/ 以 -D_POSIX_C_SOURCE=200809L -D_DARWIN_C_SOURCE 编译。
linux	posix/，等待用 linux/ 中的 epoll。
posix	只有 POSIX.1-2008，以 -D_POSIX_C_SOURCE=200809L 编译，用 poll 等待，但有两个例外：随机字节来自 /dev/urandom，一个 POSIX 没有提到的文件；fsync 来自 POSIX 可选的文件同步（File Synchronization）。适用于任何其它 Unix。
win32	Winsock 2 和 Windows API，32 位与 64 位，从 Windows Vista 的 API 起，用 WSAPoll 等待。程序要链接 ws2_32 和 bcrypt。
freertos	FreeRTOS+TCP，用它的套接字集合等待。需要 RTOS_INCLUDES，以及第 7.4 节中板子的调用。

续页

building-zh.tex Page 8 November 8, 2026 13:38

表 3.1 各系统(续)

OS	它构建什么
ucos3	uC/OS-III 加 uC/TCP-IP, 用 NetSock_Sel 等待。需要 RTOS_INCLUDES, 以及第 7.4 节中板子的调用。
none	不要 platform/ 里的任何东西: 只有 src/, 因此没有 TLS(第 7 章)。

OS 指不出文件夹时,构建还没开始就停下:

```
$ make OS=beos
Makefile:...: *** OS=beos names no folder of platform/: use posix, linux, macos,
win32, freertos or ucos3, or none.  Stop.
```

3.1 从本机构建另一个系统

自带目标和 C 库的编译器,旁边什么都不装也能为另一个系统构建。用 zig:

```
# 只有 POSIX, 用本机的编译器。
make OS=posix

# x86-64 上的 Linux, 分别用 glibc 和 musl。
make OS=linux CC='zig cc -target x86_64-linux-gnu'  AR='zig ar' PAGE_SIZE=4096
make OS=linux CC='zig cc -target x86_64-linux-musl' AR='zig ar' PAGE_SIZE=4096

# Windows, 64 位和 32 位。
make OS=win32 CC='zig cc -target x86_64-windows-gnu' AR='zig ar' PAGE_SIZE=4096
make OS=win32 CC='zig cc -target x86-windows-gnu'    AR='zig ar' PAGE_SIZE=4096
```

给出 AR 是因为 zig cc 报不出归档器(§2.1);给出 PAGE_SIZE 是因为构建的机器的页不是运行程序的机器的页(§2.2)。

4

内存函数

编译器会为 memcpy、memmove、memset 和 memcmp 发出调用，即使代码一个也没调用：结构体赋值、大的初始化器和复制用的循环，都可能变成其中之一。src/ 什么都不调用，但没有 C 库时，总得有东西来满足这些调用。PORT 指定 src/port/ 下做这件事的文件夹，并把它构建进 libterae_port.a：

- **这个归档**。它定义那四个函数，在 Arm 上还定义运行时 ABI 的 __aeabi_mem* 名字。程序把它紧接在 libterae.a 之后链接，这样库的调用、程序的调用和编译器的调用都由它来满足。
- **库**。指定了汇编文件夹时，src/ 以 -DMQTT_PORT_MEMORY 编译，复制改用该文件夹的 memcpy（用它的另一个名字 mqtt_port_memcpy），而不是自己那个逐字节的循环。

4.1 各文件夹

表 4.1 给出每种处理器对应的文件夹。它就是 PORT=auto 取的那个文件夹，所以可以问出来，而不必查（§4.2）。

表 4.1 src/port/ 的各文件夹

处理器	文件夹
Cortex-M3、M4、M7、M33 和 M55：带 Thumb-2 的 M 系列	armv7m
ARM7TDMI、ARM9、ARM11、Cortex-R（TMS570、RM4x），以及大端 Cortex-A	armv4t
32 位 Cortex-A，小端，允许非对齐载入	arm
带 Advanced SIMD 的 AArch64	aarch64

续页

building-zh.tex Page 10 November 8, 2026 13:38

表 4.1 src/port/ 的各文件夹(续)

处理器	文件夹
x86-64(不含 x32);以及 i386	x86_64, i386
RISC-V, 32 位(ESP32-C3 和 C6、SHAKTI E);带 M 的 64 位	riscv32, riscv64
带 o32 ABI 的 MIPS32, 不适用于 microMIPS(PIC32)	mips
带 ELFv2 的 64 位 PowerPC	powerpc64
带 MOVW 的 AVR, 不适用于 AVRtiny	avr
从 SH-2 起的 SuperH、带字符串指令的 RX、68000 和 ColdFire, 以及 Xtensa(ESP32、S2、S3)。已汇编或构建, 但尚未运行	sh, rx, m68k, xtensa
其余一切: Cortex-M0、M0+ 和 M23, MSP430, 带 -mstrict-align 的 AArch64, 以及所有不是 ELF 的目标, 如 Windows 和 macOS	generic

generic 是那四个函数的 C 版本，一次一个字节，任何处理器都能跑。它以 -ffreestanding -fno-builtin 编译，用 GCC 时还加 -fno-tree-loop-distribute-patterns，以免某个循环被变成对它所处的那个函数的调用。它不定义 MQTT_PORT_MEMORY，因为它的 memcpy 本来就是 src/ 会跑的那个循环。

与处理器不匹配的文件夹，会在它自己的汇编里让构建停下，错误信息说明该文件夹需要什么：例如在 Cortex-M0 上用 PORT=armv7m 时，信息说需要带 Thumb-2 的 M 系列 Arm。不是文件夹的名字会让构建停下，并列出那十六个：

```
$ make PORT=sparc
Makefile:...: *** PORT=sparc names no folder of src/port/: use auto, or one of
aarch64 arm armv4t armv7m avr generic i386 m68k mips powerpc64 riscv32 riscv64
rx sh x86_64 xtensa.  Stop.
```

4.2 PORT=auto 怎样决定

PORT=auto 用构建自己的 CC 及其 CFLAGS 跑 src/port/select.in：那是一组对编译器所定义宏的测试，预处理器从里面留下某个文件夹的名字。因此它必须看到真实构建的编译器和选项，而 make print-port 什么都不构建就显示出它挑了哪个：

building-zh.tex Page 11 November 8, 2026 13:38


```
$ make print-port PORT=auto CROSS_COMPILE=arm-none-eabi- \
      CFLAGS='-mcpu=cortex-m4 -mthumb'
armv7m
$ make print-port PORT=auto CROSS_COMPILE=arm-none-eabi- \
      CFLAGS='-mcpu=cortex-r4f -mbig-endian'
armv4t
$ make print-port PORT=auto CROSS_COMPILE=arm-none-eabi- \
      CFLAGS='-mcpu=cortex-m0 -mthumb'
generic
```

这些测试按下面的顺序进行，auto 没取到预期的文件夹时，原因就在其中：

- **不是 ELF**。Windows、macOS 以及所有不是 ELF 的目标都取 generic：每个汇编文件夹都是 ELF 的。
- **AArch64**。需要 __ARM_NEON 和 __ARM_FEATURE_UNALIGNED，所以 -mstrict-align 得到 generic。
- **x86**。x86_64 需要 64 位指针，所以 x32 得到 generic；其后是 i386。
- **Arm**。带 Thumb-2 的 M 系列得到 armv7m；从 Armv7 起、小端且允许非对齐载入的 A 系列得到 arm；其它同时有 Arm 和 Thumb 指令集的 Arm 得到 armv4t。
- **RISC-V**。按 __riscv_xlen 分开；64 位需要 __riscv_mul。
- **其余**。MIPS 需要 o32 且不是 microMIPS，PowerPC 需要 64 位 ELFv2，AVR 需要 MOVW 且不是 AVRtiny，SuperH 不能是 SH-1 或 SH-5，RX 必须允许字符串指令。然后是 m68k 和 xtensa；剩下的都是 generic。

编译器不在 PATH 上，或拒绝 CFLAGS 中的某个选项（例如 GCC 拒绝 -mcpu=cortex-r5f，它写作 cortex-r5）时，构建就停下，并说 PORT=auto couldn't read src/port/select.in。用 -E -P -x c 和同样的选项手工对该文件跑一遍编译器，就能看到它自己的信息。

4.3 只要内存归档

有自己构建系统或自己 C 库的项目，可能只想要 libterae_port.a。make port 只构建它，接受构建的所有变量，没给 PORT 时停下：

```
make port PORT=aarch64 CC='zig cc -target aarch64-linux-musl' AR='zig ar'
make port PORT=armv7m CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' BUILD=/path/to/firmware/terae
```

5

带 TLS 构建

TLS 是一个文件，platform/platform_tls.c，WITH_TLS=yes 时针对 Mbed TLS 3.6 构建。没有 WITH_TLS=yes 时同一个文件照样构建，mqtt_tls_available 返回假，base_tls.h 的其它每个函数都报告 MQTT_TLS_ERROR，于是程序问一次就能决定。

5.1 Mbed TLS

Mbed TLS 不在仓库里；测试用的版本是 3.6.7。它用 libterae 的配置 platform/mbedtls_config.h 构建一次，配置文件用绝对路径给出，因为 Mbed TLS 里有一个同名文件，相对路径会先找到它：

```
URL=https://github.com/Mbed-TLS/mbedtls/releases/download
curl -LO "$URL/mbedtls-3.6.7/mbedtls-3.6.7.tar.bz2"
tar -xjf mbedtls-3.6.7.tar.bz2
cmake -S mbedtls-3.6.7 -B mbedtls-build -DCMAKE_INSTALL_PREFIX="$HOME/mbedtls" \
      -DMBEDTLS_USER_CONFIG_FILE=/path/to/libterae/platform/mbedtls_config.h \
      -DENABLE_PROGRAMS=OFF -DENABLE_TESTING=OFF -DMBEDTLS_FATAL_WARNINGS=OFF
cmake --build mbedtls-build && cmake --install mbedtls-build
```

该配置让 Mbed TLS 从 platform_tls.c 里的静态池取内存，而不是从 malloc 取，这样 TLS 保持了库不用分配器的承诺。做到这一点的两个设置是 MBEDTLS_PLATFORM_MEMORY 和 MBEDTLS_MEMORY_BUFFER_ALLOC_C。

building-zh.tex Page 14 November 8, 2026 13:38

5.2 库与程序

```
make WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls"      # build/tls/libterae.a

cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U main.c build/tls/libterae.a \
   "$HOME/mbedtls/lib/libmbedtls.a" "$HOME/mbedtls/lib/libmbedx509.a" \
   "$HOME/mbedtls/lib/libmbedcrypto.a" -o program
```

这时 platform/ 的文件以 -DMQTT_WITH_TLS 编译，Mbed TLS 的头文件来自 MBEDTLS_PREFIX(编译器自己找得到时可以不给)，并把 platform/mbedtls_config.h 的绝对路径作为 Mbed TLS 的配置。程序不需要任何 TLS 定义：只有 platform_tls.c 读取这样的定义。程序先链接 libterae.a，然后 Mbed TLS 的三个归档，每个都排在调用它的那个之后。

5.3 板子上的 TLS

板子为它自己的处理器构建 Mbed TLS，用同样的配置，并给池一个板子 RAM 容得下的大小。池为每个会话装一整份 Mbed TLS 上下文及其缓冲区，所以这两个数字要一起定：

```
make WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls-cortex-m4" \
     OS=freertos PORT=auto CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' RTOS_INCLUDES='-I...' PAGE_SIZE=4096 \
     TLS_SESSIONS=2 TLS_POOL_BYTES=131072      # build/tls-freertos-armv7m/
```

6

共享库

程序也可以从共享库取得 libterae：程序启动时，由系统的加载器找到它。`make shared` 在 `lib/` 里构建一个，这个文件夹仓库并不保存。固件链接的、测试用的，仍然是归档；共享库是给 Linux、macOS、Windows 或别的 POSIX 系统上的程序用的：由另一个项目构建的程序，从另一种语言调用该库的程序，或者不重新链接就要换上新版本库的程序。

6.1 生成什么

```
make shared                                          # lib/
make shared WITH_TLS=yes MBEDTLS_PREFIX="$HOME/mbedtls"   # lib/tls/
make shared OS=win32 CC='zig cc -target x86_64-windows-gnu' PAGE_SIZE=4096
                                                     # lib/win32/
```

表 6.1 共享库（1.0.5 版）

系统	文件	旁边还有
linux、posix	`libterae.so.1.0.5`	`libterae.so.1`，即程序记录的名字，以及 `-lterae` 找到的 `libterae.so`：两者都链接到它。
macos	`libterae.1.dylib`	链接到它的 `libterae.dylib`。程序记录的是 `@rpath/libterae.1.dylib`。
win32	`libterae-1.dll`	导入库 `terae.lib`：GNU `ld`、`lld` 和 `zig` 用 `-lterae` 都找得到它，微软的链接器则按名字取用。

每个文件夹里还有 `pkgconfig/libterae.pc`（§6.2）。目标文件为它重新编译一遍，编

building-zh.tex Page 16 November 8, 2026 13:38

成可在任何地址运行的代码，即 -fPIC，Windows 不需要它。它们放在 BUILD 里的 shared 文件夹中，所以 libterae.a 保持原样。

lib/ 下的文件夹按构建在 build/ 下的文件夹命名（§2.4）：lib/tls、lib/win32、lib/tls-linux。给了 BUILD 时，库放进它里面的 lib 文件夹，这样换了页大小的构建就不会顶替默认的那个。LIBDIR 可以指定任何别的文件夹。make clean 把库连同它的构建一起删掉，但从不删除传给 make 的 LIBDIR，因为那里可能还放着别的库。

6.2 程序链接什么

```
export PKG_CONFIG_PATH="$PWD/lib/pkgconfig"
cc -std=c99 $(pkg-config --cflags libterae) main.c $(pkg-config --libs libterae) \
   -o program
```

libterae.pc 记下了程序需要从这次构建知道的东西：版本，指向 include/ 的 -I，库构建时用的页大小和各表大小，以及 -L 和 -lterae。在 Linux 和 macOS 上，它还给程序一条指向该文件夹的运行时路径，让加载器在那里找到库，因为加载器自己不会去仓库里的文件夹找。不用 pkg-config 时，同样的命令是：

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U main.c -Llib -lterae \
   -Wl,-rpath,"$PWD/lib" -o program
```

程序不再链接别的东西：共享库已经链接了 Mbed TLS（带 TLS 的构建），在 Windows 上还链接了 ws2_32 和 bcrypt。Windows 没有运行时路径，所以 DLL 放在程序旁边，或放在 PATH 里的某个文件夹中。

6.3 版本

该库只有一个版本号，由三个 0 到 255 的数组成，写在 include/base_library.h 里：MQTT_LIBRARY_VERSION_MAJOR、_MINOR 和 _PATCH。Makefile 从那里读取它们，所以共享库的名字不会和程序编译时用的头文件对不上；make print-version 把它们打印出来，如 1.0.5。

- 主版本。为上一版构建的程序可能要修改或重新构建才能用这一版时，它就变。程序记录的名字，libterae.so.1、libterae.1.dylib 或 libterae-1.dll，随它而变，所以程序绝不会加载另一个主版本的库。

building-zh.tex Page 17 November 8, 2026 13:38

- **次版本**。库增加了东西、而为更早的次版本构建的程序仍然能用时，它就变。macOS 把它记为库的兼容版本，加载器拒绝比程序链接时更旧的库。
- **修订版本**。程序看得到的东西都没变时，它就变。

页大小不在名字里，但它是程序和库共有的一部分（§2.2）。可能遇到别处构建的库的程序，在启动时用它自己头文件里的数字问一下：

```
if (!mqtt_library_compatible(MQTT_LIBRARY_VERSION, MQTT_PAGE_SIZE)) {
    /* 主版本不同、次版本更新，或页大小不同。 */
    return 1;
}
```

6.4 它给程序什么

程序看得到名字以 `mqtt` 开头的函数，也就是 `include/` 里的全部函数。`platform/` 的其余部分和 Mbed TLS 都留在里面：ELF 系统把一次调用绑定到它找到的第一个同名函数上，否则程序里一个叫 `platform_wait_call` 的函数就会顶替库自己的那个。在 ELF 系统上，构建为链接器写一份清单，即 `BUILD` 的 `shared` 文件夹里的 `exports.map`，GNU `ld`、`gold`、`lld` 和 `mold` 都接受它。在 macOS 上，命令行上的一个模式起同样的作用。DLL 列出它的每一个函数，这无害：Windows 把每次调用绑定到它指名的那个 DLL。

6.5 带 TLS

共享库从 `MBEDTLS_PREFIX/lib` 链接 `-lmbedtls -lmbedx509 -lmbedcrypto`，所以那必须是用 `platform/mbedtls_config.h` 构建的 Mbed TLS（第 5 章）。这样它的归档就进了共享库。在 Linux 和其它 ELF 系统上，这些归档也必须是位置无关的：在配置 Mbed TLS 的那条 `cmake` 命令里加上 `-DCMAKE_POSITION_INDEPENDENT_CODE=ON`。macOS 和 Windows 什么都不用加。

6.6 哪里没有

`make shared` 拒绝没有加载器的系统，即 `OS=freertos`、`ucos3` 或 `none`，因为固件链接的是 `libterae.a`。它也拒绝 `PORT:PORT` 的 `memcpy` 和 `memset` 是给没有 C 库的程序用的，而共享库总是和一个 C 库一起加载，它们会在整个程序里顶替 C 库的那些函数。

7

嵌入式系统

在板子上，构建取三种形态之一：

表 7.1 嵌入式构建的形态

形态	构建什么，程序做什么
裸机构建，OS=none	• 只有 src/，以及 PORT 的内存函数。 • 程序搬运字节、提供时间，通过 base_server.h 和 base_client.h：*_receive_space 与 *_receive_commit 接收到达的数据，*_transmit_view 与 *_transmit_consume 给出要发送的数据，*_tick 驱动定时器。
内核，OS=freertos 或 OS=ucos3	• src/ 和 platform/，架在内核自己的网络栈上。 • 程序在一个任务里调用 mqtt_net_server_run 或 mqtt_net_client_step，并定义第 7.4 节中板子的调用：platform/ 用来输出的控制台、随机字节，以及保存存储区的闪存。
POSIX 层，OS=posix	与桌面相同，用于有 POSIX 的小系统，或页为 4 KiB 的 Linux。

7.1 裸机构建

用 GNU 工具链时，先建归档，再建程序：

```
make OS=none PORT=auto CROSS_COMPILE=arm-none-eabi- \
     CFLAGS='-mcpu=cortex-m4 -mthumb -O2' PAGE_SIZE=4096
                    # build/none-armv7m/: libterae.a 和 libterae_port.a

arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os -nostdlib -Wl,--gc-sections \
     -T link.ld -Iinclude -DMQTT_PAGE_SIZE=4096U startup.c main.c \
     build/none-armv7m/libterae.a build/none-armv7m/libterae_port.a -lgcc \
     -o firmware.elf
```

building-zh.tex Page 20 November 8, 2026 13:38

CFLAGS 指名处理器，因为工具链的默认不是板子的。link.ld 和 startup.c 是板子的。归档的顺序就是静态链接需要的顺序：先是库，然后是它调用的内存函数，最后是 -lgcc，它给出编译器自己的辅助函数，例如 64 位除法。

用 clang 时，目标是 CC 的一部分，另外三样要手工给：能写 ELF 的归档器、用来链接的 lld，以及编译器的辅助函数——Apple 的 clang 对 Arm 一个也没有。GNU 的 libgcc 可以用，为目标构建的 compiler-rt 也可以：

```
make OS=none PORT=auto CC='clang --target=thumbv7m-none-eabi' AR=llvm-ar \
     CFLAGS='-mcpu=cortex-m4 -O2' PAGE_SIZE=4096

clang --target=thumbv7m-none-eabi -mcpu=cortex-m4 -O2 -nostdlib -fuse-ld=lld \
      -Wl,--gc-sections -T link.ld -Iinclude -DMQTT_PAGE_SIZE=4096U \
      startup.c main.c build/none-armv7m/libterae.a \
      build/none-armv7m/libterae_port.a \
      "$(arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -print-libgcc-file-name)" \
      -o firmware.elf
```

不加 -fuse-ld=lld 时，macOS 上的 clang 会把 Arm 的目标文件交给系统的链接器，后者回答 ld: unknown options: -Bstatic -EL。

7.2 程序提供什么

- 内存。src/ 不分配任何东西。服务器使用的每个数组都取自交给 mqtt_server_init 的 mqtt_server_storage，大小由 MQTT_PAGE_SIZE 和 base_server.h 中的计数决定，客户端也是如此。
- 时间。每个需要时间的调用都接收 now_ms，一个只增不减的 64 位毫秒计数；板子的 32 位节拍计数由程序扩展。
- 输出。库什么都不打印，platform/ 只通过板子的 *_board_write 打印。
- 闪存。在实时内核上，会话状态的存储区保存在板子指定的一块闪存区域中；用 OS=none 时，则由程序自己保存。

7.3 实时内核

OS=freertos 和 OS=ucos3 需要内核的头文件、网络栈的头文件以及程序的配置，全都在 RTOS_INCLUDES 里，只给该系统文件夹的文件编译时使用。内核本身由程序构建并链接：

building-zh.tex Page 21 November 8, 2026 13:38


```
make OS=freertos PORT=auto CROSS_COMPILE=arm-none-eabi- PAGE_SIZE=4096 \
     CFLAGS='-mcpu=cortex-m3 -mthumb -O2 -ffunction-sections -fdata-sections' \
     RTOS_INCLUDES="-I$FREERTOS/include -I$FREERTOS/portable/GCC/ARM_CM3 \
                    -I$PLUS_TCP/source/include \
                    -I$PLUS_TCP/source/portable/Compiler/GCC -I$BOARD"
```

构建写进 build/freertos-armv7m/。$BOARD 所指的文件夹中放着两个配置文件，FreeRTOSConfig.h 和 FreeRTOSIPConfig.h。uC/OS-III 用 OS=ucos3 以同样的方式构建。两个系统都已经设好表 2.2 中那些较小的尺寸；想要别的尺寸的板子，在命令行上给出，并同样给它的程序。

7.4 板子提供什么

在 OS=freertos 和 OS=ucos3 上，内核没有的东西来自程序所用的板子，以程序定义的函数的形式给出，它们声明在 platform/freertos/freertos_internal.h 和 platform/ucos3/ucos3_internal.h 中。链接 platform/ 的循环的程序需要全部这些函数，因为循环会走到存储区的代码，而后者要调用它们。

- **输出。**freertos_board_write 或 ucos3_board_write：控制台，供 platform/ 打印。
- **随机字节。**xApplicationGetRandomNumber，FreeRTOS+TCP 本来就要求每个程序提供它；或者 ucos3_board_random。
- **存储区的闪存。**表 7.2 中的七个调用，各以 freertos_board_store_ 或 ucos3_board_store_ 开头，做到了所要求的事时返回 true。没有闪存可给存储区的板子也要定义它们，并让 _region 返回 false，这样就不会打开任何存储区。

building-zh.tex Page 22 November 8, 2026 13:38

表 7.2 板子的闪存调用

调用	作用
`_region`	找出保存某个名字的存储区的区域：它的编号、它的大小（整数个擦除块），以及一个块的大小。
`_read`	读取一个区域中的字节。
`_program`	写入一段已擦除的字节，不论闪存自己的写入单位是多少。
`_erase`	擦除一个区域中的整个块。
`_sync`	让写入的内容经受得住断电，用于会把写入暂时扣住的闪存或其控制器。
`_set_aside`	在存储区擦除一个区域之前，把它保存的东西以自己的编号保存到别处；板子无处可存时返回 false：什么都不会丢失。
`_remove`	删除一个区域以及从它移到一旁的东西，供测试使用。

闪存被看作 NOR 闪存那样：擦除把一个块的每个字节都设为 0xFF，写入只能清除已擦除字节中的位。

7.5 固件的编译选项

表 7.3 固件的编译选项

选项	它做什么

`-ffunction-sections`
`-fdata-sections`
`-Wl,--gc-sections`（链接时）
	每个函数和数据各占一节，只保留固件用得到的那些。

`-fno-unwind-tables` `-fno-asynchronous-unwind-tables`	不生成展开堆栈用的表，C 固件从不读它们。
`-nostdlib` 和 `-lgcc`	不要板子没有的启动文件和 C 库，但要编译器的辅助函数，那是必需的。
处理器的：`-mcpu`、`-mthumb`、`-mfloat-abi`、`-mfpu`	库、它的 port 文件夹和程序要用同样的选项，否则它们的调用对不上。

8

另一个 C 库

库对 C 库要求很少。src/ 只包含 stddef.h、stdint.h 和 stdbool.h，每个编译器都有。platform/posix/ 只调用 POSIX.1-2008 有的东西，编译时只定义 _POSIX_C_SOURCE，所以有这些函数的 C 库就能把它构建出来。有两样东西超出了 POSIX.1-2008 的基本部分：来自其可选的文件同步的 fsync，以及它没有提到的文件 /dev/urandom。

8.1 platform/posix/ 需要什么

- **头文件**。errno.h、fcntl.h、limits.h、netdb.h、netinet/in.h、netinet/tcp.h、poll.h、signal.h、string.h、sys/socket.h、sys/stat.h、time.h 和 unistd.h。
- **套接字**。socket、bind、listen、accept、connect、send、sendmsg、recvmsg、setsockopt、socketpair、fcntl、close、getaddrinfo 和 freeaddrinfo。
- **等待、时间与信号**。poll、pipe、read、write、clock_gettime、sysconf、sigaction 和 sigemptyset。
- **文件**。open、pread、pwrite、ftruncate、fstat、lstat、mkdir、link、unlink、rmdir、fsync 和 time，用于保存在文件中的存储区；open 也用于 /dev/urandom。
- **字符串**。memcpy、memset 和 strlen。
- **进程**。fork、alarm、clock 和 _exit，藏在 mqtt_platform_fork、mqtt_platform_interrupt_after_s 和 mqtt_platform_cpu_us 之后。它们在一个单独的文件里，所以静态链接只把它们带进调用这些函数的程序。

  没有 malloc，没有 printf，也没有线程。

building-zh.tex Page 24 November 8, 2026 13:38

8.2 让编译器指向它

C 库是 CC 的事，而 CC 可以带任何参数：整个 C 库的 sysroot，或只给它头文件的包含路径。

```
# 一个完整的 C 库，这里是 musl，通过自带它的编译器使用。
make OS=posix CC='zig cc -target x86_64-linux-musl' AR='zig ar' PAGE_SIZE=4096

# 只用某个 C 库的头文件，除编译器的以外不要别的。
make OS=posix PAGE_SIZE=4096 \
     CC="cc -nostdinc -isystem $HOME/my-libc/include \
         -isystem $(cc -print-file-name=include)"
```

-nostdinc 藏起其它所有 C 库，第二个 -isystem 再把编译器自己的头文件（如 stdint.h）放回来。用 -isystem 而不是 -I，可以避免在带 -Werror 的构建里冒出 C 库的警告。

有自己 C 库的项目，或者自带那四个内存函数、因此不给 PORT，或者用 make port 取它们（第 4 章），并把 libterae_port.a 链接在它的 C 库之前，这样它的定义就能满足每一个调用。

8.3 检查一次构建调用了什么

一个归档向 C 库要的，是它的成员留下未定义的名字，减去其它成员定义了的那些。用 macOS 或 GNU 的 nm：

```
nm -u -j build/posix/libterae.a | grep -v -e ':$' -e '^$' | sort -u > needed
nm -g -U -j build/posix/libterae.a | grep -v -e ':$' -e '^$' | sort -u > defined
comm -23 needed defined
```

对一次 OS=posix 构建，那就是上面那份清单，再加上编译器自己添的东西：在 macOS 上，为把内存置零的 memset 而出现的 bzero，errno 背后的函数，以及——因为 Apple 的 clang 打开了栈保护——__stack_chk_fail 和 __stack_chk_guard。没有这些的 C 库，要在 CFLAGS 中加 -fno-stack-protector 来构建。

一个新的 C 库，先构建一遍，再看清单里没有哪个名字它缺少，就算验过了。

9

参考

9.1 目标

- **make、make all**。libterae.a，给了 PORT 时还有 libterae_port.a。
- **make port**。只有 libterae_port.a。
- **make print-port**。auto 决定之后 PORT 取的文件夹；没有 PORT 时什么都不输出。
- **make shared**。共享库和 libterae.pc，放在 lib/ 或按构建命名的 lib/ 下的文件夹里（第 6 章）。
- **make print-version**。库的版本，如 1.0.5（§6.3）。
- **make clean**。删掉 BUILD，以及这次构建会用的 lib/ 下的文件夹，除非给了 LIBDIR。

9.2 构建什么时候停下

表 9.1 失败是什么意思

信息	原因，以及怎么办
OS=... names no folder of platform/	系统名拼错了：信息里列出了各文件夹。
PORT=... names no folder of src/port/	文件夹名拼错了，或者是处理器的名字：见表 4.1。
PORT=auto: ... couldn't read src/port/select.in	编译器不在 PATH 上，或拒绝了一个选项（§4.2）。

续页

building-zh.tex Page 26 November 8, 2026 13:38

表 9.1 失败是什么意思(续)

信息	原因,以及怎么办
`OS=none builds no platform/, where TLS is`	TLS 是 `platform/` 里的文件:选一个系统,或者不要 TLS。
`make shared: OS=... has no loader`	板子链接的是 `libterae.a`:不要 `shared`。
`make shared: PORT is for a program with no C library`	共享库和一个 C 库一起运行:不带 `PORT` 构建它。
`include/base_library.h gives no version`	`MQTT_LIBRARY_VERSION_MAJOR`、`_MINOR` 或 `_PATCH` 的定义缺了,或者不是一个数字后面跟 `U`。
`MQTT_PAGE_SIZE must be from 4096 to 65536`,或 `must be a power of two`	`PAGE_SIZE` 不对,在构建里或在程序的 `-DMQTT_PAGE_SIZE` 里。
`MQTT_PLATFORM_WAIT_SOCKETS must be from 8 to 1048576`,或 `MQTT_NET_TLS_SESSIONS must be from 1 to 65536`	命令行上给的尺寸超出范围。
`FreeRTOS.h` 或 `os.h` 找不到	缺 `RTOS_INCLUDES`,或其中没有内核、网络栈或板子的配置。
`mbedtls/ssl.h` 找不到	`WITH_TLS=yes` 却没给 `MBEDTLS_PREFIX`,而 Mbed TLS 不在编译器查找的地方。
链接时 `mqtt_port_memcpy` 未定义	库是用 `PORT` 构建的:把 `libterae_port.a` 紧接在 `libterae.a` 之后链接。
裸机链接时 `memset` 或 `memcpy` 未定义	库构建时没有 `PORT`,又没东西提供它们:带上 `PORT` 重新构建,或链接一个 C 库。
`ranlib: warning: ... not a mach-o file`,随后是归档里有的符号却未定义	本机的归档器为另一个目标写了归档,却没有写索引:在 `AR` 中指名目标的那个。
`ld: unknown options: -Bstatic -EL`	`clang` 把 Arm 的目标文件交给了系统的链接器:加上 `-fuse-ld=lld`。
`undefined symbol: __aeabi_uldivmod`	裸机链接没带编译器的辅助函数:加上 `-lgcc`,或 `compiler-rt`。
`make shared WITH_TLS=yes` 时出现 `relocation R_X86_64_32 ... recompile with -fPIC`	Mbed TLS 的归档不是位置无关的:加上 `-DCMAKE_POSITION_INDEPENDENT_CODE=ON` 重新构建它们。

续页

building-zh.tex Page 27 November 8, 2026 13:38

表 9.1 失败是什么意思(续)

信息	原因,以及怎么办
`Library not loaded: @rpath/libterae.1.dylib`, 或 `libterae.so.1: cannot open shared object file`	程序没有指向 `lib/` 的运行时路径: 用 `-Wl,-rpath,<folder>` 链接它, 就像 `libterae.pc` 做的那样。

9.3 在别的项目的构建里使用

把 libterae 放进固件的项目,构建它时把每个变量都写出来,这样在任何装有该工具链的机器上构建都相同:

```
TERAE_BUILD := $(abspath build/terae)

$(TERAE_BUILD)/libterae.a: FORCE
^^I$(MAKE) -C libterae BUILD=$(TERAE_BUILD) OS=freertos PORT=armv7m \
^^I        CROSS_COMPILE=arm-none-eabi- CFLAGS="$(BOARD_CFLAGS)" \
^^I        RTOS_INCLUDES="$(BOARD_INCLUDES)" PAGE_SIZE=4096
FORCE:
```

BUILD 是绝对路径,因为 `make -C` 会先换文件夹。这条规则每次都跑,由内层 make 判断什么过时了。程序用同样的 `-DMQTT_PAGE_SIZE=4096U` 编译。桌面上的项目要共享库时,也这样要 shared,然后在 `$(TERAE_BUILD)/lib` 里找到它和它的 `libterae.pc`。